

Boggle Solver Code Review Summary Report

The goal of this project was to implement a Python-based Boggle solver capable of identifying all valid words within an $N \times N$ grid by connecting adjacent tiles without reuse. The approach centered on an efficient Depth-First Search (DFS) with backtracking, supported by optimized data structures including a set for $O(1)$ dictionary lookups and a prefix set for "early-exit" pruning. This optimization significantly reduces the search space by terminating exploration as soon as a letter sequence can no longer form a valid word, while robust input validation ensures the system remains stable when encountering malformed data types or non-square dimensions.

The feedback I receive highlights my solver's efficiency and robustness as standout strengths, specifically praising the use of a prefix set for early pruning and the thorough handling of edge cases like empty or non-square grids. While the reviewer found my code's organization and separation of concerns to be well-structured, they suggested enhancing the documentation through more frequent inline comments to clarify the underlying logic for future contributors. Additionally, they made note of formatting and consistency, such as standardizing spacing and docstrings, I was recommended to provide the script with a more polished, professional finish.

Overall, their code is strong with key strengths being the implementation of a Trie (Prefix Tree) data structure, which provides high-performance word validation, and the input sanitization that ensures case-insensitive matching for both the grid and dictionary. This approach is very scalable, but to further improve their code, I suggested considering addressing the logic redundancy between the constructor and the setter methods by centralizing the cleaning logic, and optimize the DFS by passing the TrieNode through the recursive calls.

To align the code with professional standards, I performed a static analysis and applied several PEP 8 style corrections, focusing primarily on improving "scannability" and long-term maintainability. This involved standardizing all indentation to a consistent 4-space format, inserting proper vertical spacing between class definitions and the `main()` function, and adding horizontal inline spacing around operators and after commas for better visual clarity. Additionally, I integrated concise docstrings at the start of each method to define their purpose and reorganized the internal comments to ensure the logic is easy for any developer to follow at a glance.

Static Analysis Results

Category	Issue Found	Fix Applied
Indentation	Inconsistent use of 2-space and 4-space indents.	Standardized all indentation to 4 spaces per PEP 8.
Naming & Spacing	Missing whitespace around operators (e.g., <code>i+1</code>) and after commas.	Added spaces to improve readability (e.g., <code>range(len(word))</code> and <code>i + 1</code>).
Documentation	Methods lacked docstrings or high-level explanations.	Added Google-style docstrings and detailed inline comments for the backtracking logic.
Blank Lines	Insufficient vertical spacing between class methods and the <code>main</code> block.	Inserted two blank lines before the <code>main()</code> function and between class definitions.

To ensure code stability after refactoring, I conducted a series of regression tests comparing the updated implementation against a "Gold Standard" baseline from the original script. By executing both versions with a standard 4x4 grid, I verified that the refactored code produced identical, alphabetically sorted results—including correctly handling multi-character tiles like "Qu" as single units. Beyond basic functionality, I performed edge-case re-testing to confirm that the program gracefully handles malformed inputs, such as empty grids, non-list dictionary types, and non-square N x N dimensions, by returning an empty list rather than triggering a crash.

I learned that professional workflows rely on Continuous Integration (CI) tools like GitHub Actions. By setting up a workflow, the system automatically checks my code for errors the moment I push it. This prevents "broken" code from ever reaching the final product. Just because code runs on my local machine (or in Codio) doesn't mean it's perfect. Testing against a "Gold Standard" baseline and running a remote test suite ensures that the code is portable and meets the specific requirements of the project.